

SLEZSKÁ UNIVERZITA V OPAVĚ
Filozoficko-přírodovědecká fakulta v Opavě

BAKALÁŘSKÁ PRÁCE

SLEZSKÁ UNIVERZITA V OPAVĚ
Filozoficko-přírodovědecká fakulta v Opavě

Ondřej Marek

Studijní program: Moderní informatika
Specializace: Informační a komunikační technologie

**Kontejnerizace aplikace s využitím technologie
Docker**

**Application Containerization using Docker
Technology**

Bakalářská práce

Opava 2024

Vedoucí bakalářské práce:
RNDr. Šárka Vavrečková, Ph.D.

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

Akademický rok: 2023/2024

Zadávací ústav: Ústav informatiky

Student: Ondřej Marek

UČO: 60651

Program: Moderní informatika

Specializace: Informační a komunikační technologie

Téma práce: Kontejnerizace aplikace s využitím technologie Docker

Téma práce anglicky: Application Containerization using Docker Technology

Zadání: Student se ve své práci bude věnovat problematice kontejnerizace aplikací. Vysvětlí princip kontejnerizace včetně rozdílů oproti běžné virtualizaci, výhod a nevýhod, popíše a srovná několik běžně používaných technologií pro kontejnery a jejich orchestraci, a dále se bude podrobněji věnovat technologii Docker. V praktické části se autor zaměří na demonstraci přenositelnosti řešení s využitím Dockeru.

Literatura: Docker Docs [online]. Docker [cit. 2023-12-06]. Dostupné na: <https://docs.docker.com/>
Bentaleb, Ouafa et al. Containerization technologies: taxonomies, applications and challenges. The Journal of Supercomputing, 78(1). 2022. ISSN 1573-0484. Dostupné na: <https://link.springer.com/article/10.1007/s11227-021-03914-1>
Souppaya, Murugiah, John Morello a Karen Scarfone. Application Container Security Guide. Draft NIST Special Publication 800-190. NIST, 2017. Dostupné také z: <https://csrc.nist.gov/csrc/media/publications/sp/800-190/draft/documents/sp800-190-draft.pdf>
Grubor, S. Deployment with Docker: Apply continuous integration models, deploy applications quicker, and scale at large by putting Docker to work. Packt Publishing Ltd, 2017. ISBN 9781786463227. Část knihy dostupná online na <https://books.google.cz/books?id=lkBPdWAAQBAJ&printsec=frontcover>

Vedoucí práce: RNDr. Šárka Vavrečková, Ph.D.

Datum zadání práce: 11. 12. 2023

Souhlasím se zadáním (podpis, datum):



.....
doc. RNDr. Luděk Cienciala, Ph.D.
vedoucí ústavu

Abstrakt

Tato práce se zaměřuje na kontejnerizaci aplikací pomocí platformy Docker. V úvodní části práce jsou podrobně vysvětleny základy kontejnerů a rozdíly mezi kontejnerizací a virtualizací. Dále jsou čtenáři představeny nejpoužívanější kontejnerové enginy a orchestrátory. V další části následuje detailní popis technologie Docker, včetně jeho architektury a ukázky příkazů pro manipulaci s obrazy a kontejnery. Nakonec jsou zde popsány bezpečnostní prvky Dockeru. Poslední část práce se zabývá konkrétní aplikací, pro niž jsou vytvořeny kontejnery. Zde je dokumentován proces vytvoření a sestavení obrazů a následné spuštění jednotlivých kontejnerů. Nakonec je popsána automatizace pomocí nástroje Docker Compose.

Klíčová slova

Kontejner, kontejnerizace, Docker, orchestrátor, bezpečnost

Abstract

This thesis focuses on containerizing applications using the Docker platform. The introductory part of the thesis provides a detailed explanation of the basics of containers and the differences between containerization and virtualization. Furthermore, the most commonly used container engines and orchestrators are introduced to the reader. The next section provides a detailed description of the Docker technology, including its architecture and examples of commands for manipulating images and containers. Finally, the security features of Docker are described. The last part of the thesis deals with a specific application for which containers are created. Here, the process of creating and building images and subsequently running individual containers is documented. Finally, the automation using the Docker Compose tool is described.

Keywords

Container, containerization, Docker, orchestrator, security

Čestné prohlášení

Prohlašuji, že jsem tuto práci vypracoval samostatně. Veškerou literaturu a další zdroje, z nichž jsem při zpracování čerpal, v práci řádně cituji a jsou uvedeny v seznamu použité literatury.

V Opavě dne 15. dubna 2024

.....

Ondřej Marek

Poděkování

Rád bych poděkoval za odborné vedení, rady a cenné poznatky k danému tématu vedoucí práce RNDr. Šárce Vavrečkové, Ph.D. Také bych rád poděkoval mé rodině a přátelům za podporu a pomoc během mého studia.

Obsah

Úvod	1
1 Kontejner a kontejnerizace	2
1.1 Principy kontejnerizace	2
1.1.1 Agilita	2
1.1.2 Přenositelnost	3
1.1.3 Rychlost a škálovatelnost	3
1.2 Kontejnerizace vs. virtualizace	4
1.3 Kontejnerové enginy	5
1.3.1 Docker	5
1.3.2 Podman	6
1.3.3 CRI-O a containerd	7
1.4 Orchestrace kontejnerů	7
1.4.1 Docker Compose	8
1.4.2 Docker Swarm	8
1.4.3 Kubernetes	9
1.4.4 Platformy postavené na Kubernetes	10
2 Technologie Docker	11
2.1 Architektura Docker enginu	11
2.1.1 Docker Daemon	12
2.1.2 containerd	13
2.1.3 runc	14
2.1.4 Docker Client	14
2.1.5 Docker Host	14
2.1.6 Docker Registry	15
2.1.7 Docker Objects	15
2.2 Práce s obrazy a kontejnery	17
2.2.1 Dockerfile	17
2.2.2 Základní příkazy	18
2.3 Bezpečnost v Dockeru	20
2.3.1 Namespaces	20
2.3.2 Control Groups	20
2.3.3 Zranitelnosti Docker Daemonu	21
2.3.4 Capabilities linuxového jádra	21
2.3.5 Další zabezpečení	22

3	Kontejnerizace aplikace s využitím Dockeru	23
3.1	Popis aplikace	23
3.2	Instalace Dockeru	24
3.2.1	Instalace Dockeru pro operační systém Windows a Mac OS . .	24
3.2.2	Instalace Dockeru pro operační systém Ubuntu	26
3.3	Kontejnerizace frontendové části aplikace	27
3.3.1	Vytvoření Dockerfile	27
3.3.2	Sestavení obrazu a spuštění kontejneru	29
3.4	Kontejnerizace backendové části aplikace	29
3.4.1	Vytvoření Dockerfile	29
3.4.2	Sestavení obrazu a spuštění kontejneru	30
3.5	Automatizace sestavení obrazů a spuštění kontejnerů	31
3.6	Ukázka instalace kontejnerizované aplikace	33
	Závěr	34
	Seznam použité literatury	35
	Seznam obrázků	38
	Seznam zkratk	39
	Přílohy	40

Úvod

Téma práce „Kontejnerizace aplikace s využitím technologie Docker“ jsem si vybral s ohledem na stále rostoucí význam kontejnerizace v moderním softwarovém vývoji a provozu aplikací. V dnešní době, kdy je klíčové optimalizovat procesy vývoje, nasazení a správy aplikací, se kontejnerizace stala nezbytným nástrojem pro vývojáře a provozovatele softwaru. Chtěl bych prostřednictvím této práce lépe porozumět principům a možnostem kontejnerizace a konkrétněji se seznámit s platformou Docker a jejím využitím pro správu a provoz aplikací. Díky této práci si kladu za cíl nejen teoreticky prozkoumat tuto problematiku, ale také ji aplikovat v praxi.

Hlavním cílem této práce je detailně přiblížit čtenářům široké možnosti a koncepty spojené s technologií kontejnerů a jejich praktické využití pomocí platformy Docker. Bude se zaměřovat nejen na teoretické základy a principy kontejnerizace, ale také na praktické aspekty, jako je správné nastavení a konfigurace kontejnerů spuštění aplikací v kontejnerizovaném prostředí.

První část této práce se věnuje základním principům kontejnerizace a zdůrazňuje výhody, které tato technologie přináší. Kromě agility, přenositelnosti, rychlosti a škálovatelnosti je zde kladen důraz na porovnání s tradiční virtualizací, což pomáhá lépe porozumět kontejnerizaci jako takové. Dále jsou představeny nejpopulárnější kontejnerové enginy a orchestrace kontejnerů.

Druhá část práce se zaměřuje na technologii Dockeru, což je jeden z předních nástrojů v oblasti kontejnerizace. Čtenář je proveden architekturou Docker enginu a detailně se seznámí s jednotlivými komponentami tohoto prostředí. Dále je popsán proces práce s obrazy a kontejnery pomocí Dockerfile a základních příkazů. Nakonec se práce zaměřuje na bezpečnostní aspekty Dockeru a strategie pro zajištění spolehlivosti a ochrany prostředí kontejnerizace.

Poslední část práce přináší konkrétní aplikaci teoretických poznatků v praxi. Zde je popsán postup vytvoření kontejneru pro konkrétní aplikaci, včetně tvorby Dockerfile, sestavení Docker obrazů a kontejnerů a jejich následné spuštění pomocí Docker Compose. Je zde kladen důraz na demonstraci přenositelnosti a flexibility Dockeru, díky které je možné spustit aplikace z jakéhokoliv prostředí.

1 Kontejner a kontejnerizace

V dnešní době, kdy se technologie neustále vyvíjí, hledají organizace efektivní a flexibilní způsoby nasazování a správy svých aplikací. V této snaze vystupují kontejnery jako klíčový prvek, který nabízí izolaci, snadné nasazení a konzistentní prostředí pro spuštění aplikací.

1.1 Principy kontejnerizace

Kontejner je standardizovaná jednotka softwaru, která balí kód a všechny jeho závislosti, jako je běhové prostředí, systémové knihovny a nastavení, takže aplikace běží rychle a spolehlivě v různých počítačových prostředích. [6] Kontejnery jsou založeny na konceptu kontejnerizace, což je technologie umožňující zabalení aplikace a všeho, co potřebuje k běhu, do jednoho balíčku, tedy kontejneru. Každý kontejner sdílí jádro operačního systému s hostitelským strojem.

Při tradičním vývoji je kód vyvíjen v konkrétním prostředí, pokud je ale přenesen do jiného, mohou nastat různé chyby a problémy. Při využití kontejnerizace se kód společně se všemi potřebnými závislostmi zabalí do kontejneru, je „oddělen“ od operačního systému. Díky tomu je dosaženo vysoké míry přenositelnosti a kód se chová identicky bez ohledu na platformu, na které běží, ať už se jedná o lokální počítač, cloud, nebo specifické prostředí. I když kontejnerizace zjednodušuje přenos kódu, stále záleží na kompatibilitě s jádrem hostitelského operačního systému. Samotná aplikace uvnitř kontejneru je opravdu „oddělena“ od hostitelského operačního systému, ale kontejner samotný není. Například Windows kontejnery nelze spustit na Linuxu. Mnoho kontejnerizačních platforem ale umožňuje vytvořit a spouštět Linuxové kontejnery i v jiných operačních systémech. [9]

1.1.1 Agilita

Jedním z hlavních pilířů kontejnerizace je podpora agility. Agilita v kontextu kontejnerů znamená schopnost rychle a účinně reagovat na měnící se vývojové požadavky a podmínky. Kontejnery umožňují vývojářům provádět rychlé aktualizace aplikací, což je klíčové pro agilní vývoj. Rozdělení závislostí umožňuje aktualizaci konkrétního kontejneru bez výrazného ovlivnění ostatních částí aplikace nebo infrastruktury.

tury. Funkční flexibilita je dalším aspektem agility v kontextu kontejnerizace, která umožňuje snadné přidávání a odstraňování funkcí, což výrazně usnadňuje modulární přístup k vývoji. Agilita také podporuje rychlost nasazení a lehkost. Konzistence kontejnerů zase umožňuje rychlé nasazení změn v produkčním prostředí, což zkracuje čas potřebný k nasazení nových funkcí nebo oprav. [9]

1.1.2 Přenositelnost

Dalším důležitým principem je přenositelnost aplikací mezi různými prostředími. Přenositelnost v kontejneru znamená, že kontejnery umožňují aplikacím pokračovat v běhu nezávisle na operačním systému (samotné kontejnery musí být opět kompatibilní s jádrem hostitelského operačního systému). Standardizované uživatelské rozhraní, které kontejnery poskytují běžícím aplikacím, zajišťuje konzistentní provoz ve všech prostředích. Tato funkce usnadňuje přesun aplikací z vývoje přes testování do výroby. Díky nezávislosti na infrastruktuře lze aplikaci použít v jakékoli infrastruktuře, která podporuje kontejnery, ať už se jedná o cloudovou službu, vlastní serverovou farmu nebo hybridní prostředí. [9]

1.1.3 Rychlost a škálovatelnost

Kontejnery jsou často označovány jako „lightweight“, což znamená, že sdílejí jádro operačního systému stroje a nepotřebují přebytné náklady spojené s přidružením operačního systému ke každé aplikaci. Taková jednoduchost nejen zvyšuje efektivitu serverů, ale také snižuje náklady na servery a licence, zrychluje dobu spouštění, protože není třeba spouštět celý operační systém.

Škálovatelnost je klíčovým prvkem kontejnerizace, který umožňuje dynamicky reagovat na různé situace, jako jsou zátěžové situace nebo nárůst požadavků. Díky této funkci lze přidat více kontejnerů a zajistit tak optimální využití dostupných zdrojů. Tímto způsobem lze účinně využívat zdroje a minimalizovat náklady, což je nejen ekonomicky výhodné, ale také zvyšuje celkovou produktivitu infrastruktury. [9]

1.2 Kontejnerizace vs. virtualizace

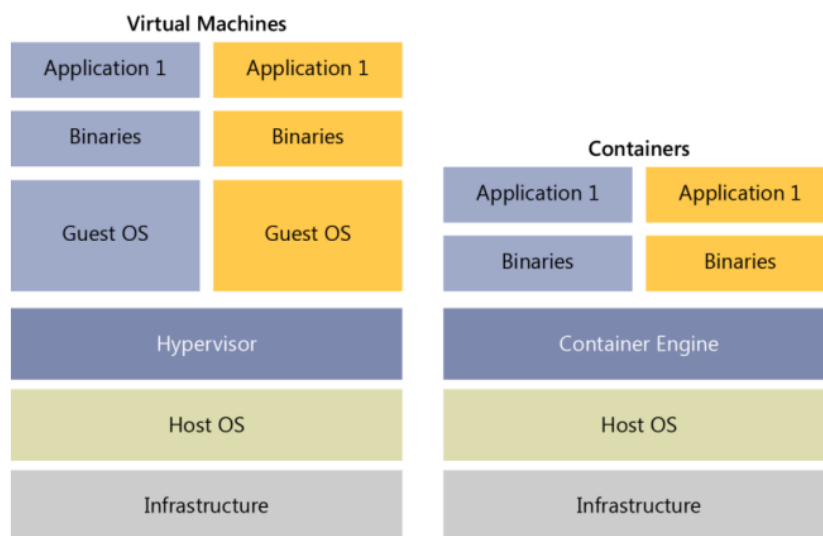
Po představení základních principů kontejnerizace se zaměříme na srovnání s tradiční virtualizací. Oba přístupy jsou navrženy tak, aby izolovaly a spravovaly aplikace, ale liší se v několika klíčových aspektech.

Virtualizace je tradiční metoda, při které virtuální stroje (VM) běží na hostitelském hardwaru. Každý virtuální počítač obsahuje kompletní operační systém včetně jádra, přidružených souborů, knihoven, závislostí a simulovaného fyzického hardwaru. Vytváří izolované prostředí, kde lze na jediném fyzickém počítači provozovat různé operační systémy. Jednou z hlavních výhod virtualizace je vysoká izolace mezi virtuálními stroji, která zajišťuje bezpečnost a izolaci prostředí. Každý virtuální stroj má však plný operační systém, což přináší další náklady a spotřebovává více systémových prostředků. Virtualizace obvykle používá hypervizory, což jsou speciální vrstvy softwaru, které umožňují běh více operačních systémů na jednom fyzickém počítači.

Naopak kontejnerizace využívá sdílení jádra operačního systému hosta mezi kontejnery. Každý kontejner sdílí jádro s hostitelským systémem, ale běží v izolovaném prostoru s vlastními závislostmi a procesy. Oproti VM kontejnery nezahrnují kopii operačního systému a jsou tedy mnohem efektivnější a méně náročné na údržbu, i větší aplikace lze spustit během několika sekund.

Jak už jsem zmínil, kontejnery jsou „lightweight“. To znamená, že mají menší spotřebu systémových zdrojů než virtuální stroje a vyžadují kratší dobu spuštění, což umožňuje spustit na stejné výpočetní kapacitě mnohem více kontejnerů než jeden virtuální stroj. [9]

Obrázek 1 ukazuje, že v případě virtualizace na každém stroji běží operační systém. Tyto operační systémy řídí hypervizor, což je speciální softwarová vrstva, která umožňuje běh více operačních systémů na jednom fyzickém stroji. Na druhé straně kontejnery mají sdílený operační systém a na hostitelském počítači je nainstalován kontejnerový engine, což je aplikace nebo nástroj, který spravuje životní cyklus kontejnerů. [14]



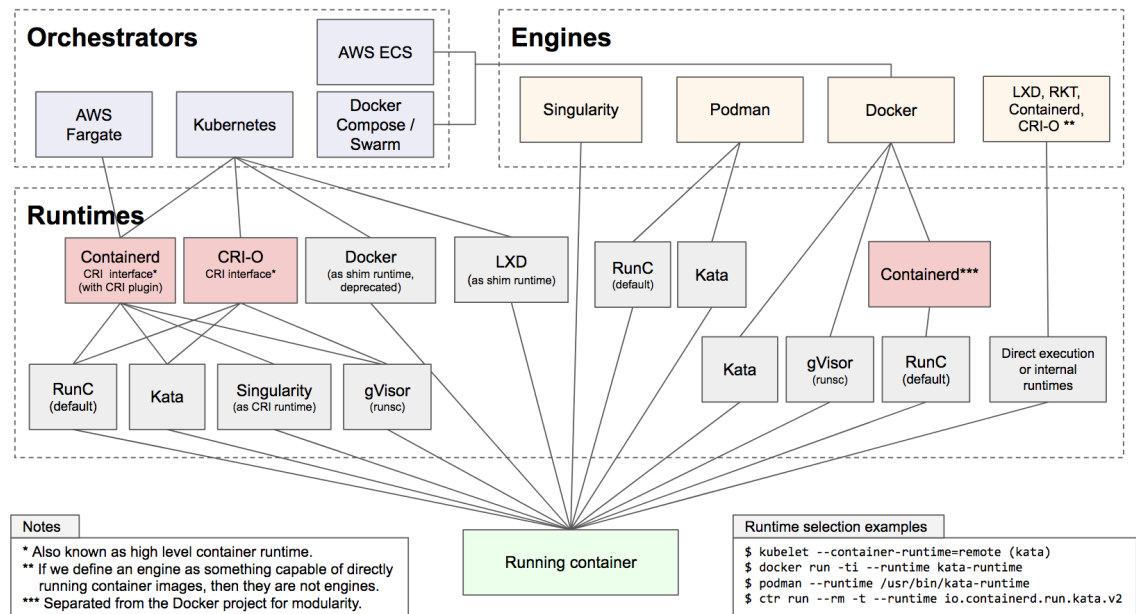
Obrázek 1: Porovnání VM a kontejnerizace [14]

1.3 Kontejnerové enginy

Kontejnerové enginy jsou klíčovými nástroji, které spravují a umožňují běh kontejnerů. Jde o software, která přijímá požadavky uživatelů, včetně možností příkazového řádku, získává kontejnerové obrazy a z pohledu koncového uživatele spouští kontejner. To ale není tak jednoduché, se samotným kontejnerem doopravdy pracují takzvané kontejnerové runtime, které jsou součástí kontejnerového enginu nebo kontejnerového orchestrátoru, ty budou popsány v následující kapitole. Tyto runtime mají také často stejný název jako enginy, jak lze vidět na obrázku 2 na straně 6. Kontejnerový engine umožňuje běh samotných kontejnerů, zatímco orchestrátory umožňují koordinaci a správu více kontejnerů, aby pracovaly společně jako jedna aplikace. V této práci se budu zaměřovat na Docker, ten ale není jediným kontejnerovým enginem, v této kapitole budou podrobněji popsány některé významné kontejnerové enginy. [22]

1.3.1 Docker

Docker je jedním z nejpopulárnějších a nejrozšířenějších kontejnerových enginů. Poprvé byl uveden v roce 2013 a rychle se stal standardem v oblasti kontejnerizace. Docker používá svůj vlastní formát kontejnerů a je postaven na architektuře klient-server. Jeho hlavními komponentami jsou Docker Daemon (dále démon), který běží



Obrázek 2: Diagram vztahů mezi orchestrátory, enginy a runtime [22]

na pozadí, a rozhraní Docker CLI, které umožňuje interakci s démonem. Tento démon spravuje a koordinuje běh kontejnerů a je odpovědný za komunikaci s ostatními systémovými komponentami. Ačkoli démon poskytuje pokročilé funkce pro správu kontejnerů, jeho nevýhodou je provoz na úrovni operačního systému s privilegovanými přístupovými právy. To znamená, že má přístup k systémovým prostředkům a funkcím, které mohou vést ke zneužití.

Výhodou Dockeru je právě jeho popularita, má rozsáhlou dokumentaci, velkou komunitu a repozitář, Docker Hub, kde je možné sdílet, vyhledávat a stahovat obrazy kontejnerů. Docker umožňuje orchestraci kontejnerů pomocí nástrojů jako je Docker Compose a Kubernetes. [2, 22]

1.3.2 Podman

Podman je alternativní kontejnerový engine vyvinutý zejména pro Linux. Jednou z hlavních odlišností od Dockeru je, že Podman nemá démona (daemonless). Každý kontejner je spuštěn v izolovaném procesu, což znamená, že není potřeba běžícího démona na pozadí. Tato vlastnost usnadňuje integraci s nástroji pro správu procesů a zajišťuje vyšší bezpečnost. Podman je také schopen běžet jako neprivilegovaný

vaný uživatel, což znamená, že není vyžadováno oprávnění superuživatele pro správu kontejnerů. To je důležité z hlediska bezpečnosti a izolace. [19, 22]

1.3.3 CRI-O a containerd

CRI-O i containerd nejsou přímo určeny k použití jak kontejnerové enginy, jde spíše o již zmíněný runtime. Containerd poskytuje základní runtime funkcionalitu, zatímco CRI-O je optimalizováno pro běh kontejnerů v prostředí Kubernetes. I tak lze oba použít jako engine, CRI-O lze ovládat pomocí nástroje crictl (CRI Control). Tato utilita slouží zejména pro účely ladění (debugging) a testování. Containerd může být ovládán pomocí nástroje nazývaného ctr (Containerd CLI), který poskytuje uživatelské rozhraní pro interakci s Containerd. S ctr lze spouštět, zastavovat a spravovat kontejnery. [22]

1.4 Orchestrace kontejnerů

V dnešní době, kdy jsou vyvíjeny a nasazovány stále složitější aplikace, je pro správu prostředí kontejnerů zapotřebí efektivní a robustní řešení. Základní kontejnerový engine, jako je například Docker, poskytuje skvělý nástroj pro izolaci a spouštění kontejnerů. Nicméně s nárůstem rozsahu a komplexnosti aplikací se stává evidentní, že je až nereálné pro optimální řízení kontejnerů využívat pouze engine. Zde vstupuje do hry kontejnerová orchestrace. Oproti kontejnerovým enginům, které se zaměřují na jednotlivé kontejnery, orchestrátor poskytuje sadu nástrojů a funkcí pro správu a automatizaci vícekontejnerových prostředí. Tímto způsobem je řízeno nasazování, škálování, monitorování a aktualizace rozsáhlých mikroslužebních architektur.

Orchestrace kontejnerů představuje automatizaci mnoha operačních úkonů nezbytných pro běh kontejnerizovaných služeb. To zahrnuje celou škálu činností od provozování a nasazování po škálování, správu sítí, vyvažování zátěže a další aspekty.

Jednou z nejdůležitějších výhod je provozní zjednodušení, což znamená účinnou správu kontejnerů v distribuovaném prostředí. Další předností je odolnost, kterou tyto nástroje poskytují pomocí automatického restartování nebo škálování kontejnerů, což výrazně zvyšuje odolnost aplikací vůči selháním.

Další výhodou, kterou orchestrace kontejnerů nabízí, je zvýšená bezpečnost kontejnerizovaných aplikací. Tohoto bezpečného přístupu je dosaženo pomocí automatizace, která minimalizuje nebo eliminuje lidskou chybu přispívající ke zranitelnosti systému. Automatická reakce na problémy je také díky automatizaci mnohem rychlejší, než kdyby se musely řešit ručně. [10, 22]

1.4.1 Docker Compose

Docker Compose je nástroj určený pro definování a spouštění více kontejnerů jako jediné aplikace. Umožňuje uživatelům popsat konfiguraci svých služeb v textovém souboru pomocí syntaxe YAML. Tento soubor obsahuje informace o tom, jaké kontejnery budou spuštěny, jaké porty budou otevřeny, jaké sítě budou vytvořeny a další konfigurační prvky potřebné pro správné nasazení aplikace.

Jednou z výhod Docker Compose je možnost definovat a spustit celou aplikaci jediným příkazem, což zjednodušuje proces vývoje a testování. Díky tomu mohou vývojáři rychle vytvářet a nasazovat své aplikace bez složité konfigurace prostředí.

Docker Compose také umožňuje vývojářům definovat různé konfigurace pro vývoj a produkci. To znamená, že vývojáři mohou simulovat produkční prostředí na svých počítačích s konfigurací, která se blíží realitě, a minimalizovat tak riziko chyb při nasazení.

Je to snadný orchestrátor pro použití a je užitečný pro lokální a menší nasazení. Nicméně Docker Compose není navržen pro správu a škálování kontejnerů v prostředí s vysokým provozem nebo pro komplexní distribuované systémy. Pro tyto účely jsou vhodnější robustnější orchestrátory jako je Docker Swarm nebo Kubernetes, které poskytují pokročilejší funkcionality pro řízení a monitorování kontejnerů v produkčním prostředí. [2, 22]

1.4.2 Docker Swarm

Docker Swarm je orchestrátor kontejnerů vyvinutý jako součást ekosystému Docker. Jeho hlavním cílem je poskytnout snadno použitelné řešení pro správu a nasazování kontejnerizovaných aplikací. Architektura Docker Swarm je zjednodušená ve srov-

nání s Kubernetes, což se projevuje jednotným API a menším počtem komponent pro nasazení. Tento nástroj se využívá pro menší a středně velká nasazení.

Ve srovnání s Kubernetes je Docker Swarm považován za snazší při konfiguraci a používání. Takže implementace kontejnerů je rychlejší bez velké administrativní složitosti. Podporuje možnost horizontálního škálování, což umožňuje uživatelům přidávat další instance (například kontejnery) k již existujícím. Tento postup následně vede k rovnoměrnému rozložení celkové zátěže mezi tyto jednotlivé instance.

Docker Swarm má však ve srovnání s Kubernetes menší komunitu a ekosystém. To může ovlivnit dostupnost služeb podpory a počet dostupných nástrojů třetích stran. [2, 22]

1.4.3 Kubernetes

Další orchestrátor je Kubernetes, který vznikl jako open-source a byl vyvinutý původně společností Google. Kubernetes je navržen pro řešení složitějších požadavků v oblasti správy a orchestrace kontejnerů. Kubernetes poskytuje pokročilé možnosti pro škálování, monitorování a řízení stavu aplikací.

Kubernetes je nezávislý na konkrétní kontejnerové platformě, což znamená, že není vázán pouze na Docker kontejnery. Tím pádem může být využíván s různými typy kontejnerů. Díky velké komunitě, ekosystému nástrojů a aplikací třetích stran existují pro uživatele dostatečné možnosti a podpora.

Kubernetes je používán pro vysoce škálovatelné řešení a složitější aplikace. Podporuje možnost jak horizontálního, tak vertikálního škálování. Vertikální škálování poskytuje možnost přidávat prostředky do jediné existující instance (jako je kontejner), čímž se snižuje zatížení konkrétní instance. Kromě toho disponuje pokročilými nástroji pro monitorování a logování, rovněž umožňuje správu stavů aplikací na vyšší úrovni. [11, 22]

1.4.4 Platformy postavené na Kubernetes

V kontejnerové orchestraci existuje řada specializovaných platforem, které využívají základů populárního Kubernetes a poskytují dodatečné funkce a nástroje pro správu kontejnerizovaných prostředí.

OpenShift, postavený na Kubernetes, poskytuje nástavbu, která usnadňuje vývoj, testování a provoz kontejnerizovaných aplikací. Má přehledné rozhraní a schopnost automatické škálovatelnosti, používá se pro správu mikroslužeb a moderních aplikací.

Google Kubernetes Engine (GKE), jako plně řízená služba od Google Cloud, obsahuje výhody jednoduchého nasazení a správy kontejnerů pomocí Kubernetes. Integrace s dalšími nástroji od Google umožňuje komplexní řízení cloudových aplikací.

Azure Kubernetes Service (AKS), nabízená společností Microsoft, umožňuje snadnou orchestraci kontejnerů v rámci ekosystému Azure. Integrace s dalšími nástroji od Microsoftu poskytuje uživatelům sadu nástrojů pro monitorování, správu a škálování aplikací.

Rancher, jako open-source platforma, přináší správu Kubernetes klastrů bez zbytečné složitosti. Usnadňuje monitorování a řízení kontejnerů v prostředí Kubernetes, čímž zjednodušuje práci vývojářům a správcům systémů. [15]

2 Technologie Docker

Docker engine je základní software, který spouští a spravuje kontejnery. Nejčastěji se však o této technologii mluví jako o Dockeru. [20] Tato platforma pomáhá při psaní, testování a nasazování kódu. Docker poskytuje tyto služby:

Rychlé a konzistentní doručování aplikací: Docker zjednodušuje životní cyklus vývoje tím, že umožňuje vývojářům poskytovat aplikace a služby ve standardizovaném prostředí pomocí místních kontejnerů. Kontejnery se staly klíčovou technologií pro implementaci principů kontinuální integrace (CI) a průběžného doručování (CD). Umožňují vytvářet izolované a konzistentní prostředí pro vývoj, testování a nasazení softwaru.

Responzivní nasazení a škálování: Dockeru umožňuje podle potřeb přenášet a škálovat pracovní zátěž. Kontejnery mohou běžet na notebooku vývojáře, na fyzickém nebo virtuálním počítači v datovém centru, na cloudovém serveru nebo ve vestavěném prostředí. Tato schopnost přenositelnosti a flexibility výrazně rozšiřuje možnosti nasazení aplikací a služeb.

Spouštění více úloh na stejném hardwaru: Docker se vyznačuje nízkou náročností na zdroje a vysokou rychlostí, což poskytuje ekonomicky efektivní alternativu k virtualizačním strojům založeným na hypervizoru. Tato technologie umožňuje efektivní využívání serverových zdrojů.

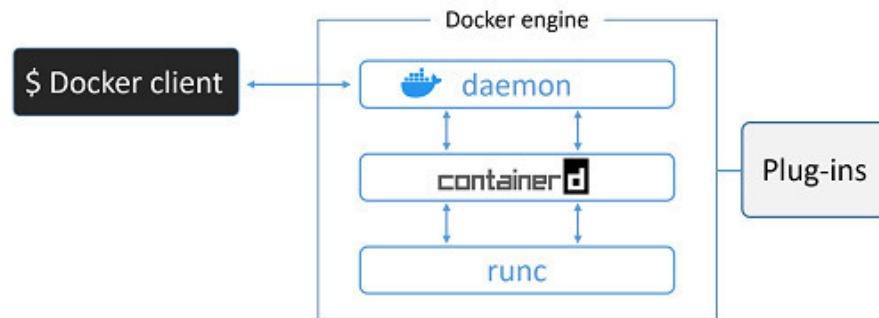
V této kapitole bude podrobněji popsána architektura Docker engine společně se základními pojmy a příkazy. [2]

2.1 Architektura Docker engine

Docker engine používá svůj vlastní formát kontejnerů, je modulární, postavený z několika malých specializovaných komponent. Hlavními komponenty jsou Docker daemon, systém pro sestavení, containerd, runc a další různé pluginy, společně vytvářejí a spravují kontejnery.

Docker engine je postaven na architektuře klient-server. Klient komunikuje s Docker daemonem, který se staral o vytváření, provoz a distribuci kontejnerů. Dnes tomu tak úplně není, pomáhá mu komponenta zvaná containerd, této komponentě zase pomáhá ještě menší komponenta runc. Jejich vzájemná komunikace je znázor-

něna na obrázku 3 na straně 12. Klient a daemon mohou běžet na stejném systému

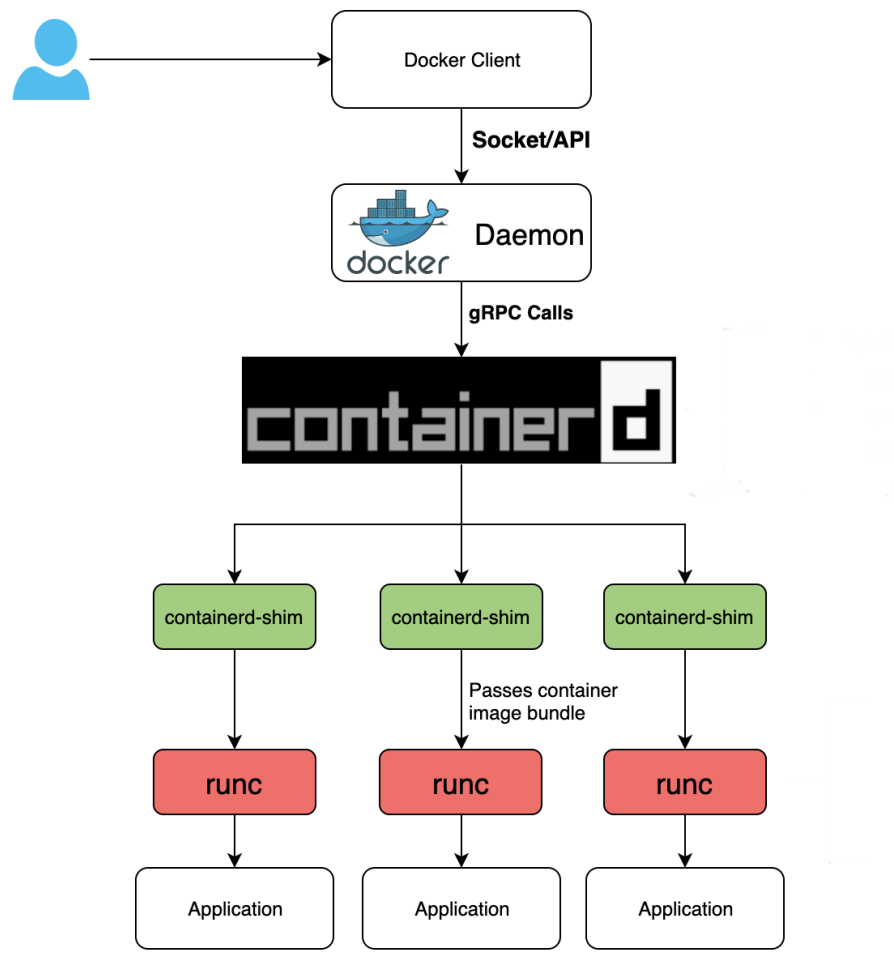


Obrázek 3: Komunikace mezi klientem, daemonem, containerd a runc [20]

nebo komunikovat vzdáleně. Komunikují mezi sebou pomocí REST API, přes UNIXové sokety nebo síťové rozhraní. Na hostitelském počítači běží samotný Docker daemon, jsou na něm uloženy obrazy kontejnerů a samotné kontejnery. [2, 20]

2.1.1 Docker Daemon

Když byl Docker engine poprvé vydán, jeho hlavní komponentou byl Docker daemon. Obsahoval veškerý kód pro API, běhové prostředí, sestavování obrazů, celkovou práci s kontejnery a další. [20] Problémem bylo, že byl monolitický, všechny jeho funkcionality byly integrovány v jediném kódu, to zpomalovalo vývoj a i samotný Docker daemon byl pomalejší. Proto se daemon začal modularizovat a některé jeho funkcionality se přesunuly do jiných speciálních nástrojů jako jsou containerd a runc, které budou popsány v následujících kapitolách. Dnes daemon neobsahuje žádný kód pro přímou práci s kontejnery. Namísto toho se jedná o komunikační prostředek mezi uživatelem a nástrojem containerd. S uživatelem daemon komunikuje pomocí Docker CLI přes API. Instrukce uživatele, jako například vytvoření kontejneru, následně předá containerd pomocí dalšího speciálního API. Výhodou odstranění veškeré logiky pro práci s kontejnery z daemona je, že lze v Docker daemonu provádět údržbu a upgrade softwaru bez žádných následků pro běžící kontejnery. V minulosti musely být během údržby zastaveny, teď už to není nutné, jedná se o takzvané “daemonless kontejnery”. [2, 13, 20]



Obrázek 4: Komunikace mezi klientem, daemonem, containerd a runc [23]

2.1.2 containerd

Tento speciální nástroj má jediný účel – řídit operace životního cyklu kontejneru. V Docker enginu se nachází mezi Docker daemonem a runc, jak lze vidět na obrázku 5 na straně 15. Později do něj byly přidány další funkce jako získávání a nahrání obrazů kontejnerů, z důvodu použití i v jiných projektech jako třeba v Kubernetes, kde je containerd často používán jako runtime. Mnohé funkce navíc jsou volitelné, nemusí být v projektu vůbec použity. Navzdory svému názvu nemůže containerd ve skutečnosti vytvářet kontejnery. K tomu používá runc. Převěd požadovaný Docker obraz na balíček OCI a řekne runc, aby ho použil k vytvoření nového kontejneru. [2, 13, 20]

2.1.3 runc

V době kdy společnost Docker Inc. pracovala na modularizaci Docker daemonu, byly definovány standardy kontejnerů, tato iniciativa se nazývá OCI (Open Container Initiative). OCI je otevřená řízená struktura, která určuje standardy formátu kontejnerů. Dnes obsahuje tři specifikace: runtime specifikaci (runtime-spec), specifikaci obrazu (image-spec) a distribuční specifikaci (distribution-spec). Od roku 2016 Docker implementuje specifikace OCI, právě od této doby se Docker daemon přímo nezabývá během kontejnerů. Tím se zabývá runc, což je referenční implementace OCI runtime-spec. Runc je nízkoúrovňový, „lightweight“ kontejnerový runtime, který má jediný úkol – vytvořit kontejner. Je hodně rychlý a jednoduchý, s kontejnery ale jinak neumí pracovat. Runc jenom čeká, než mu containerd předá potřebné instrukce pro vytvoření kontejneru, hned po vytvoření se ukončí. [8, 18, 20]

2.1.4 Docker Client

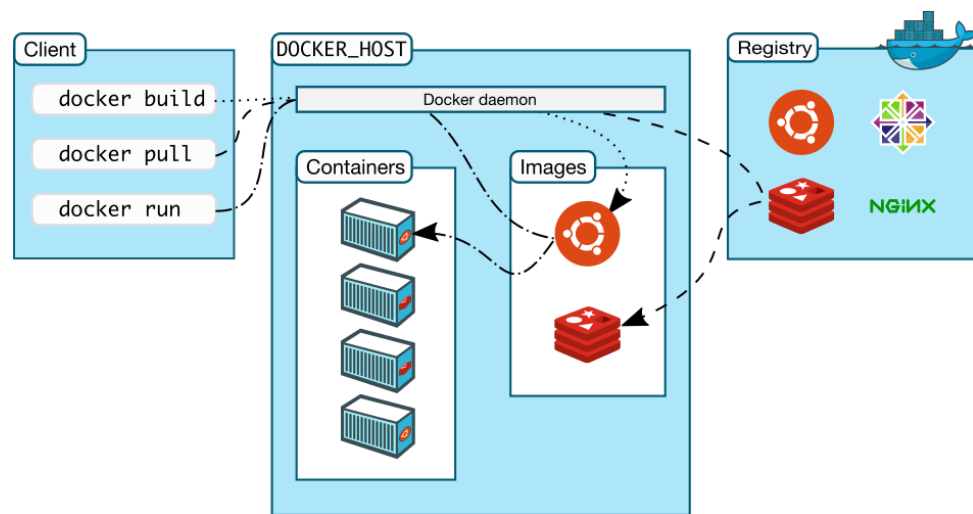
Jde o software, který obsahuje nástroje a rozhraní pro interakci s Docker daemonem. Komunikovat může například pomocí CLI nebo grafického rozhraní. Pokud uživatel zadá jakýkoliv příkaz pro vytvoření, správu nebo spuštění kontejneru, Client ho pošle přes REST API daemonovi na Docker host, který s těmito příkazy dále pracuje. Client může komunikovat s více daemony.

Nemusí běžet na výkonném počítači, slouží pouze ke vzdálenému ovládání Dockeru, veškerá práce se vykonává na Docker hostu. [2, 12]

2.1.5 Docker Host

Jde o stroj (server), na kterém běží daemon a poskytuje izolované prostředí pro běh kontejnerů a umožňuje jim sdílet zdroje stroje, jako jsou soubory, síťové rozhraní a další. Na Docker hostu může běžet několik kontejnerů najednou, přičemž každý kontejner má svůj vlastní izolovaný prostor.

Je určený k tomu, aby prováděl instrukce získané z Docker klienta, k provádění všech svých úkolů potřebuje určitý výpočetní výkon. [2, 12]



Obrázek 5: Architektura Dockeru [12]

2.1.6 Docker Registry

Registry jsou centralizovaná místa, kde se ukládají obrazy docker kontejnerů. Většina moderních registrů implementuje distribuční specifikaci OCI. Tyto obrazy lze jednoduše sdílet a lze k nim snadno přistupovat z různých prostředí.

Základním registrem je Docker Hub, což je veřejný registr poskytovaný společností Docker, který umožňuje uživatelům ukládat veřejné obrazy a také své vlastní soukromé obrazy. Veřejné obrazy lze snadno sdílet a jsou volně dostupné na použití pro všechny uživatele. V Docker Hubu existují také takzvané „oficiální repozitáře“, jde o označení repozitářů, ve kterých se nacházejí obrazy s kvalitním a aktuálním kódem, který je bezpečný a dobře zdokumentovaný. Naopak neoficiální repozitáře obsahují obrazy s neproověřeným kódem, obecně se doporučuje využívat obrazy z oficiálních úložišť. Mimo Docker Hub lze vytvořit a provozovat vlastní privátní Docker Registr. [2, 12, 20]

2.1.7 Docker Objects

Při používání Dockeru se setkáváme s různými prvky, jež tvoří jeho ekosystém. Mezi tyto objekty patří obrazy, kontejnery, sítě, svazky, pluginy a mnoho dalších objektů, které tvoří základ této technologie. V této kapitole se zaměřím na obrazy a kontejnery.

Obrazy

Obraz kontejneru je read-only šablona, která obsahuje všechny potřebné informace pro vytvoření Docker kontejnerů. Obraz je často založen na jiném obrazu s nějakými dalšími úpravami. Jeden obraz lze použít ke spuštění jednoho nebo více kontejnerů. Můžeme si vytvořit své vlastní obrazy nebo můžeme použít obrazy vytvořené jinými uživateli zveřejněné v registru.

Docker obrazy lze přirovnat k zastaveným kontejnerům. Ve skutečnosti je možné zastavit kontejner a vytvořit z něj nový obraz. Proto jsou obrazy označovány jako build-time konstruktory – sestavují kontejnery, které se zase označují run-time konstruktory – spouští aplikaci. Jakmile je kontejner spuštěn z obrazu, jsou tyto dvě struktury na sobě závislé a obraz nelze odstranit, dokud nebude zastaven a zničen poslední kontejner, který jej používal.

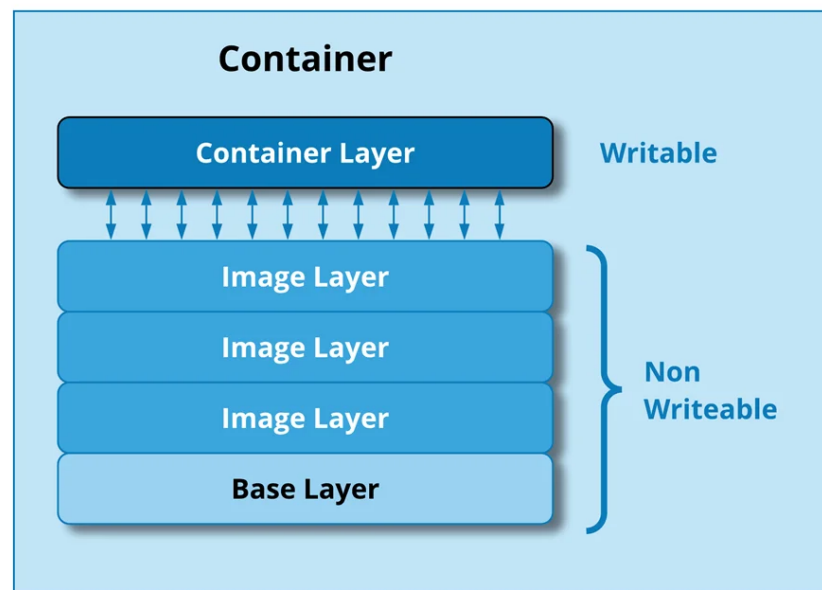
Obrazy se skládají z několika vrstev naskládaných na sebe a prezentovaných jako jeden objekt. Všechny Docker obrazy začínají základní vrstvou a postupně se do nich přidávají nové vrstvy, podle toho jak jsou prováděny změny a přidáván nový obsah. Základní vrstvou je často operační systém, v dalších vrstvách mohou být obsaženy aplikace, knihovny, a další potřebné komponenty.

Každá vrstva představuje změny v systému, které byly provedeny od předchozího stavu. Pokud se aktualizuje soubor v nějaké vrstvě, vytvoří se nová vrstva s aktualizovaným souborem, při znovusestavení obrazu se aktualizují jen ty vrstvy, ve kterých proběhla změna, s ostatními se nepracuje. To je součástí toho, co dělá obrazy „lightweight“, malé a rychlé. [1, 2]

Kontejnery

Docker kontejner je runtime prostředí s veškerými potřebnými komponentami, jako je kód, závislosti a knihovny, potřebnými k provozu kódu aplikace bez použití závislostí na hostitelském stroji. Běží na stroji, kde se nachází Docker engine.

Kontejner je spustitelná instance Docker obrazu. Stejně jako obraz, obsahuje vrstvy. Nad těmito vrstvami je však ještě jedna zapisovatelná vrstva, známá jako kontejnerová vrstva. Tato vrstva na rozdíl od ostatních vrstev umožňuje přístup pro čtení i zápis. Díky ní je možné provést změny na kontejneru izolovaně od ostatních



Obrázek 6: Vrstvy kontejnerů [7]

kontejnerů, založených na stejném obrazu. Pokud se však kontejner zastaví, všechny změny v této vrstvě se ztratí. Z běžícího kontejneru lze také vytvořit nový obraz, což umožňuje uchování provedených změn právě v kontejnerové vrstvě. [1, 2]

2.2 Práce s obrazy a kontejnery

V této kapitole bude čtenář seznámen s dokumentem pro sestavení obrazů. Dále se seznámí se základními příkazy pro práci s Dockerem, které umožňují vytvářet, spouštět, spravovat a odstraňovat kontejnery a obrazy.

2.2.1 Dockerfile

Docker může automaticky sestavovat obrazy tím, že čte instrukce z Dockerfile. Dockerfile je textový dokument, který obsahuje všechny instrukce, které uživatel může zavolat na příkazové řádce k sestavení obrazu. [2] Jde o různé instrukce pro stahování potřebných komponent, kopírování souborů do obrazu, nastavení prostředí, určení příkazu ke spuštění uvnitř kontejneru a mnoho dalšího. Instrukce jsou prováděny v pořadí od prvního řádku dolů. Jednotlivé příkazy jsou psány v tomto formátu:

```
# Comment
INSTRUCTION arguments
```

A toto je reálný příklad Dockerfile:

```
FROM ubuntu:latest
RUN apt-get update
CMD ["echo", "Hello, Docker!"]
```

Instrukce nejsou takzvaně „case-sensitive“, je ale pravidlem je psát velkými písmeny, aby je šlo snadno odlišit od argumentů. Tabulka 1 obsahuje často používané instrukce. [2]

Tabulka 1: Instrukce Dockerfile [2]

Instrukce	Popis
FROM	Vytvořit novou sestavovací fázi ze základního obrazu
RUN	Spouští příkazy v operačním systému kontejneru
COPY	Kopírovat soubory a adresáře do kontejneru
ENV	Nastavit proměnné v prostředí
ARG	Použít proměnné pro sestavení v čase kompilace
ENTRYPOINT	Specifikovat výchozí spustitelný soubor
WORKDIR	Změnit pracovní adresář
EXPOSE	Popsat, na kterých portech aplikace naslouchá
CMD	Nastavení výchozího příkazu, spustí se při startu kontejneru
LABEL	Přidat popisky do obrazu
VOLUME	Vytvořit svazky
MAINTAINER	Specifikovat autora obrazu

2.2.2 Základní příkazy

V této části se seznámíme s několika základními příkazy pro práci s Dockerem. Tyto příkazy umožňují uživatelům vytvářet, spravovat a provozovat kontejnery a obrazy. Následující seznam obsahuje základní používané příkazy:

- **docker build:** Tento příkaz slouží k sestavení obrazu z Dockerfile. Je základním krokem při vytváření vlastních obrazů pro aplikace.
- **docker run:** Používá se ke spuštění kontejneru z daného obrazu.

- `docker stop`: Slouží k zastavení běžícího kontejneru.
- `docker start`: Tento příkaz spouští zastavený kontejner. Je užitečný, pokud potřebujeme opětovně spustit již existující kontejner.
- `docker rm`: Používá se k odstranění zastaveného kontejneru z vašeho systému. Pomocí tohoto příkazu můžeme uvolnit prostředky a čistit prostředí.
- `docker rmi`: Tento příkaz odstraňuje obraz z vašeho lokálního úložiště.
- `docker ps`: Zobrazuje seznam běžících kontejnerů spolu s jejich informacemi, jako jsou ID, název a stav.
- `docker images`: Zobrazuje seznam všech dostupných obrazů na systému. Tento příkaz je užitečný pro prozkoumání dostupných obrazů a jejich verzí.
- `docker exec`: Spouští příkaz uvnitř běžícího kontejneru. Tento příkaz umožňuje uživatelům interaktivně komunikovat s kontejnerem a provádět různé operace.
- `docker inspect`: Poskytuje podrobné informace o kontejneru nebo obrazu, včetně konfigurace, síťových detailů, objemu a dalších metadat.
- `docker tag`: Přejmenuje existující obraz na jiné jméno a/nebo značku. Tento příkaz je užitečný při práci s různými verzemi obrazů a jejich správě.
- `docker push`: Nahraje lokální obraz do registru Dockeru, umožňující sdílení obrazů s ostatními uživateli a jejich nasazení na různé servery.
- `docker pull`: Stáhne obraz z registru Dockeru do lokálního úložiště. Tento příkaz je základním krokem při získávání obrazů pro použití v různých prostředích.

Některé příkazy podporují dodatečné možnosti, které umožňují uživatelům rozšířit jejich funkcionalitu. Například lze použít přepínač `-a` pro zobrazení všech kontejnerů (včetně zastavených) při použití příkazu `docker ps`. Podrobné informace a mnoho dalších příkazů Dockeru lze nalézt v oficiální dokumentaci [2]. V tomto textu čerpám z výše uvedeného zdroje.

2.3 Bezpečnost v Dockeru

Docker kontejnery jsou postaveny na jádru Linuxu, což přináší řadu bezpečnostních výhod. Při spouštění kontejneru pomocí příkazu `docker run` vytváří Docker sadu jmenných prostorů (namespaces) a omezovacích skupin (control groups) pro tento kontejner. Tyto mechanismy umožňují izolaci procesů a omezení jejich přístupu k systémovým zdrojům, což zvyšuje bezpečnost a minimalizuje riziko úniku dat nebo poškození hostitelského systému. V této kapitole budou podrobněji objasněny tyto mechanismy společně s dalšími bezpečnostními prvky. [2]

2.3.1 Namespaces

Jmenné prostory poskytují první a nejjednodušší formu izolace. Procesy běžící v rámci kontejneru nemohou vidět a už vůbec ne ovlivňovat procesy běžící v jiném kontejneru nebo na hostitelském systému. V každém kontejneru je zajištěna izolace síťových zdrojů. To znamená, že kontejnery nemají přístup k síťovým prostředkům ostatních kontejnerů, pokud není hostitelský systém konfigurován pro jejich propojení. Pokud však jsou specifikovány otevřené porty pro kontejnery nebo jsou použity odkazy, komunikace mezi nimi je umožněna. To znamená, že kontejnery mohou vzájemně posílat zprávy a navazovat síťová spojení, stejně jako s externími systémy. V síťové architektuře Dockeru jsou všechny kontejnery propojeny pomocí virtuálního bridge, což zajišťuje jejich spojení podobně jako fyzické stroje připojené k ethernetovému přepínači. [2]

2.3.2 Control Groups

Omezovací skupiny poskytují mnoho užitečných metrik, ale také zajišťují, že každý kontejner dostane svůj spravedlivý podíl na paměti, CPU a diskovém I/O. A co je důležitější, zabrání tomu, aby jeden kontejner způsobil pád systému vyčerpáním některých těchto zdrojů.

I přestože nechrání před tím, aby jeden kontejner pronikal do dat a procesů jiného, jsou omezovací skupiny zásadní pro obranu před určitými typy útoků, jako je například DoS. Zejména na platformách s více nájemci, jako jsou veřejné a privátní

PaaS, jsou klíčové pro zajištění stále dostupného (a výkonného) prostředí, i když se některé aplikace začnou chovat nevhodně. [2]

2.3.3 Zranitelnosti Docker Daemona

Jak už bylo zmíněno v dřívější kapitole, při spouštění kontejnerů (a aplikací) s Dockerem je nezbytné spouštět Docker Daemona. Tento démon vyžaduje oprávnění uživatele root, což vyžaduje zvýšenou pozornost k několika klíčovým aspektům.

Pouze důvěryhodní uživatelé by měli mít oprávnění ovládat Docker Daemona, protože Docker umožňuje sdílet adresáře mezi hostitelem a kontejnerem bez omezení přístupových práv. To může vést k bezpečnostním rizikům.

Dále je Daemon také potenciálně zranitelný vůči jiným vstupům, jako je načítání obrazů buď z disku pomocí příkazu `docker load`, nebo z internetu pomocí příkazu `docker pull`. Od novějších verzí jsou všechny obrazy uloženy a přistupujeme k nim pomocí kryptografických kontrolních součtů jejich obsahu.

A nakonec, pokud je Docker provozován na serveru, je doporučeno na serveru spouštět výhradně Docker a ostatní služby umístit do kontejnerů řízených Dockerem. [2]

2.3.4 Capabilities linuxového jádra

Docker ve výchozím nastavení spouští kontejnery s omezeným souborem capabilities. To znamená, že procesy uvnitř kontejnerů nemají stejně rozsáhlá oprávnění jako procesy běžící mimo kontejner. Typické servery běží s několika procesy s oprávněním root, jako jsou SSH daemon, cron, logovací daemon a další. Kontejnery jsou odlišné, protože většinu těchto úkolů zpracovává infrastruktura okolo kontejnerů. To znamená, že kontejnery obvykle nepotřebují „skutečné“ rootovské oprávnění. A proto mohou kontejnery běžet s omezeným souborem capabilities, což znamená, že „root“ uvnitř kontejneru má mnohem méně oprávnění než skutečný „root“.

Jedním z hlavních rizik při spouštění Docker kontejnerů je, že výchozí sada capabilities a připojení poskytnutých kontejneru může poskytnout neúplnou izolaci. Docker podporuje přidávání a odebrání capabilities. To může učinit Docker bezpečnějším odebráním capabilities, nebo méně bezpečným přidáním capabilities.

Nejlepší praxí pro uživatele by bylo odebrat všechny capabilities kromě těch, které jsou explicitně vyžadovány pro jejich procesy. [2]

2.3.5 Další zabezpečení

Docker Engine lze nakonfigurovat tak, aby spouštěl pouze podepsané obrazy, a to oficiální obrazy, které jsou ověřené a bezpečné. Tato funkce se nazývá ověřování podpisu důvěryhodnosti obsahu Dockeru (Docker Content Trust signature verification).

Kromě schopností jádra lze pro ochranu hosta použít i další nástroje linuxového jádra. Ve výchozím nastavení jsou povolené jenom schopnosti, to ale nezabraňuje ve využití kombinace známých systémů jako TOMOYO, AppArmor, SELinux, GRSEC. [2]

3 Kontejnerizace aplikace s využitím Dockeru

Tato kapitola se zaměřuje na kontejnerizaci názorné aplikace, která byla vytvořena pro účely této práce. Cílem je poskytnout praktického průvodce procesem kontejnerizace s využitím Dockeru, který může sloužit jako vzor pro kontejnerizaci dalších aplikací. Čtenář bude seznámen s danou aplikací a následně s postupem tvorby Docker obrazu pro každou její část pomocí souborů Dockerfile. Dále bude demonstrováno spuštění kontejnerů pro jednotlivé části aplikace a zajištění jejich běhu v izolovaném prostředí. V závěru kapitoly se zaměříme na využití kontejnerové orchestrace s Docker Compose, která umožňuje spuštění multikontejnerové aplikace s jednoduchou konfigurací. Závěrem bude popsán proces instalace a použití kontejnerizované aplikace v jakémkoliv prostředí.

3.1 Popis aplikace

Aplikace, která byla vytvořena pro demonstraci kontejnerizace v rámci této práce, nese název To-Do List a slouží jako nástroj pro správu úkolů. Jedná se o webovou aplikaci rozdělenou do dvou hlavních částí: klientské (frontend) a serverové (backend).

Frontend je část aplikace, se kterou uživatel přímo interaguje. Zahrnuje uživatelské rozhraní (UI), které zobrazuje obsah a umožňuje uživatelům provádět interakce. Backend je část aplikace, která běží na serveru a zajišťuje logiku aplikace, zpracovává požadavky uživatele, komunikuje s databází a provádí další operace nezbytné pro správné fungování aplikace. [25]

Klientská část aplikace slouží k zobrazování a správě úkolů v uživatelsky přívětivém rozhraní. Byla vyvinuta s využitím nástroje Vite, který umožňuje rychlý vývoj a podporuje šablonu frameworku React v jazyce TypeScript. Hlavní funkcí klientské části je formulář pro přidávání nových úkolů. Uživatelé zde zadávají název úkolu, jeho prioritu a volitelně datum a popis. Po odeslání formuláře se úkol uloží do serverové databáze a zobrazí se v seznamu úkolů. Kromě přidávání úkolů umožňuje aplikace také zobrazovat detaily úkolů, označení dokončeného úkolu a jejich mazání. Pro komunikaci se serverem využívá klientská část rozhraní Fetch, které umožňuje provádět HTTP požadavky na server. Data o úkolech se načítají pomocí

koncových bodů API ze serverové části. Tímto způsobem je zajištěna synchronizace dat mezi frontendem a backendem a uživatelé tak mohou interaktivně manipulovat s úkoly. [21, 26]

Serverová část aplikace zajišťuje správu dat a komunikaci s klientskou částí. Je napsána v jazyce TypeScript a využívá prostředí Node.js. Node.js je prostředí pro běh JavaScriptu na straně serveru, které umožňuje vývojářům vytvářet rychlé, škálovatelné a robustní webové aplikace. Pro vytvoření webového serveru a definici API je použit framework Express. Pro ukládání dat je použita NoSQL databáze MongoDB, která umožňuje ukládat a manipulovat s daty ve formátu dokumentů. Komunikaci s databází zajišťuje oficiální MongoDB ovladač pro Node.js, který poskytuje potřebné rozhraní. Framework Express slouží ke zpracování HTTP požadavků a definici různých koncových bodů pro manipulaci s daty. Umožňuje rychlé a efektivní vytvoření API s podporou zpracování různých typů požadavků a odpovědí.

Celá backendová část je nasazena na serveru, který naslouchá na definovaném portu a zajišťuje připojení klientů a správu jejich požadavků. V případě chyby při připojení k databázi server informuje o události a zajistí, že aplikace zůstane v chodu. [16, 17, 24]

3.2 Instalace Dockeru

Platforma Docker je dostupná na všech velkých operačních systémech, tedy Linuxu, Windows a Mac OS. Platforma Docker je navíc zcela zdarma pro osobní použití, malé firmy (do 250 zaměstnanců a 10 milionů USD ročního příjmu), vzdělávání a neziskový open source. [2] V této kapitole budou zmíněné předpoklady potřebné k instalaci aplikace Docker, společně s průběhem instalace na jednotlivých systémech. Je podstatné upozornit, že v době, kdy tato práce vzniká, byla použita aktuální verze Dockeru 25.0.3 na platformě Windows 11. Příkazy Dockeru by se ale v různých systémech neměly lišit.

3.2.1 Instalace Dockeru pro operační systém Windows a Mac OS

Minimální požadavky pro Docker na platformách Windows a Mac OS jsou výrazně vyšší ve srovnání s Ubuntu. To je způsobeno tím, že základní distribuce Dockeru je

To-Do list

Název úkolu:	Priorita:			
Dokončit důležitý projekt	Vysoká	☐	Více..	Smazat
Fotbalový zápas	Vysoká	☐	Více..	Smazat
Napsat domácí úkol	Střední	☐	Více..	Smazat
Zajít na nákup	Nizká	☐	Více..	Smazat
Posekat trávu	Nizká	☒	Více..	Smazat
Uklidit byt	Nizká	☒	Více..	Smazat

Vytvořeno v rámci bakalářské práce | © Ondřej Marek

Obrázek 7: Ukázka To-Do List aplikace

k dispozici pouze jako Docker Desktop, což je verze s grafickým uživatelským rozhraním. To znamená, že vyžaduje vyšší výkon a diskový prostor, minimálně 4 GB systémové paměti RAM. Zatímco v Ubuntu se nejčastěji používá varianta bez grafického rozhraní. Minimální podporovanou verzí operačního systému je 64bitový systém Windows 10. Pro Mac OS jsou podporovány vždy poslední tři verze systému, přičemž nejstarší z nich ztrácí podporu po vydání nové hlavní verze. U Windows je také potřeba mít možnost spuštění hardwarové virtualizace, to proto aby bylo možné spustit linuxové kontejnery na Windows. Instalace Docker Desktop je jednoduchá, stačí stáhnout instalační soubor pro příslušný operační systém a postupovat

podle pokynů v instalačním průvodci. I když je Docker Desktop aplikace s grafickým rozhraním, všechny potřebné příkazy lze spustit i z příkazové řádky. [3, 4]

3.2.2 Instalace Dockeru pro operační systém Ubuntu

V Ubuntu lze také nainstalovat Docker Desktop, pokud ale uživatel vyloženě netrvá na grafickém rozhraní, tak se moc nepoužívá. Nejčastěji se Docker instaluje pomocí takzvaného balíčkovacího systému apt. Jde o CLI, takže minimální požadavky jsou daleko menší, pouze 512 MB RAM paměti (doporučuje se 2 GB). Je vyžadován 64bitový operační systém, aktuálně jsou podporovány tyto verze:

- Ubuntu Mantic 23.10,
- Ubuntu Jammy 22.04 (LTS),
- Ubuntu Focal 20.04 (LTS).

Docker je kompatibilní s 86_64 (nebo amd64), armhf, arm64, s390x, a ppc64le (ppc64el) architekturami.

Instalace probíhá následovně:

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

V prvním kroku se aktualizuje seznam dostupných balíčků. Poté se nainstalují nezbytné certifikáty a nástroj sloužící k přenosu dat přes internet. Dále je vytvořen adresář `/etc/apt/keyrings`, který bude použit k uložení staženého GPG klíče pro Docker. Následně se stáhne a uloží GPG klíč Dockeru z URL na oficiálním uložišti. Nakonec nastavíme oprávnění pro nově vytvořený klíč.

```
echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

Zde je sestaven řetězec obsahující informace o Docker repozitáři a příslušném archivu, který je potřebný pro instalaci Dockeru v systému. Tento řetězec je

pak zapsán do souboru `/etc/apt/sources.list.d/docker.list`. Nakonec je znovu aktualizován seznam dostupných balíčků.

V posledním kroku se instalují potřebné balíčky Dockeru pomocí příkazu:

```
sudo apt-get install docker-ce docker-ce-cli \
    containerd.io docker-buildx-plugin docker-compose-plugin
```

Tento příkaz nainstaluje poslední verzi jádra Dockeru, společně s dalšími balíčky, které slouží například k práci s příkazovým řádkem Dockeru nebo k definování a správě multikontejnerových aplikací pomocí Docker Compose. [5]

3.3 Kontejnerizace frontendové části aplikace

Nyní se přesuneme na samotnou kontejnerizaci aplikace. Každá její část by měla být v samostatném kontejneru, protože tato architektura poskytuje několik klíčových výhod. První z nich je izolace. Každý kontejner poskytuje své vlastní oddělené prostředí, což minimalizuje rušení mezi jednotlivými částmi aplikace a zvyšuje celkovou bezpečnost systému.

Druhou výhodou je flexibilita. S každou částí aplikace běžící v samostatném kontejneru můžeme vyvíjet, aktualizovat a nasazovat každou část nezávisle. Tímto způsobem umožňuje kontejnerizace agilní vývoj a nasazování aplikace, jelikož změny v jedné části nemají vliv na ostatní.

Poslední výhodou je škálovatelnost. Pokud potřebujeme zvýšit výkon nebo zvládnout vyšší zátěž, můžeme jednoduše škálovat každou část aplikace samostatně. To znamená, že můžeme spustit více instancí backendové části aplikace, aby byla schopna obsloužit větší počet uživatelů, aniž by bylo nutné škálovat ostatní části aplikace. V této kapitole bude popsán proces sestavení obrazu a spuštění kontejneru pro frontendovou část aplikace.

3.3.1 Vytvoření Dockerfile

Pro sestavení obrazu je nutné vytvořit Dockerfile, ten je potřeba umístit do kořenového adresáře, v mém případě `frontend`. V mém konkrétním případě jsem využil Multi-stage build (vícestupňové sestavení), což je strategie, která se používá k optimalizaci velikosti výsledného obrazu nebo oddělení vývojového a produkčního prostředí. Tento přístup umožňuje oddělit proces sestavení aplikace do více fází, v mém

případě do dvou. První stádium obrazu obsahuje všechny potřebné závislosti k sestavení aplikace ve vývojovém prostředí. V druhém stádiu jsou použity výsledky prvního stádia k vytvoření finálního obrazu, který obsahuje pouze běžící aplikaci a minimální množství potřebných závislostí. Zde je ukázka první části Dockerfile:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

První instrukce definuje výchozí obraz, který bude stažen z Docker Hubu a použit pro sestavení aplikace. Jde o oficiální obraz Node.js verze 20.0.0. Definovat verzi je výhodné, protože máme jistotu, že nenastanou problémy způsobené zpětnou nekompatibilitou novějších verzí. Je zde také vidět, že je obraz založen na Alpine Linuxu, což je minimalistická linuxová distribuce. Klíčové je také jméno stádia `build`, které umožní odkazování na toto stádium v druhé části souboru.

Následující instrukce pro vytvoření pracovního adresáře kontejneru, kde budou prováděny všechny následující instrukce. Dále se zde zkopírují `package.json` a `package-lock.json`, to jsou konfigurační soubory, které se vytváří při inicializaci projektu Node.js. Následně se spustí instalace závislostí definovaných v `package.json`, a zkopírují se všechny zdrojové soubory aplikace do pracovního adresáře. Poslední instrukce spouští příkaz `build` definovaný také v `package.json`. Jde o příkaz, který kompiluje a sestavuje zdrojové soubory aplikace.

V první instrukci druhé části souboru se definuje výchozí obraz, který bude použit pro produkční nasazení aplikace. Pro spuštění lokálního serveru jsem použil nástroj Vite, který lze spustit v Node.js. Vite je ale nástroj pro vývojáře a nedoporučuje se používat v produkčním prostředí. Proto je použit webový server Nginx, který je lehký, stabilní a optimalizovaný pro efektivní doručování statického obsahu.

```
FROM nginx:1.24-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 3000
CMD ["nginx", "-g", "daemon off;"]
```

Následující instrukce kopírují výstupní soubory z prvního stádia do adresáře, který je výchozím adresářem pro webový server Nginx a následně je zkopírován

konfigurační soubor `nginx.conf` s konfigurací nginx serveru. Dále je definováno, že kontejner naslouchá na portu 3000. Poslední instrukce definuje výchozí příkaz, který se má spustit při startu kontejneru. V tomto případě se spouští Nginx server.

3.3.2 Sestavení obrazu a spuštění kontejneru

Před sestavením obrazu je také vhodné vytvořit soubor `.dockerignore`, kde se specifikují soubory a adresáře, které mají být ignorovány při sestavování obrazu. To pomáhá zmenšit velikost vytvořeného obrazu. V mém případě je zde vhodné zahrnout adresář `node_modules`, kde jsou uloženy všechny závislosti balíčků Node.js, ty potřebné budou ale nainstalovány při spuštění kontejneru. Nyní je možné sestavit obraz, je potřeba se přesunout do adresáře obsahujícího Dockerfile a použít příkaz `docker build`.

```
docker build -t frontend .
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
frontend	latest	6068099ff670	5 seconds ago	41.5MB

Následně je pro kontrolu použit příkaz `docker images`, který vypíše seznam vytvořených obrazů.

Pro spuštění kontejneru se používá příkaz `docker run`, s parametrem `-p` pro mapování portů.

```
docker run -d -p 3000:3000 frontend
docker ps
```

CONTAINER ID	IMAGE	STATUS	...
30168696ab53	frontend	Up 3 seconds	...

Následně je pro kontrolu použit příkaz `docker ps`, který vypíše seznam běžících kontejnerů. (výpis je zkrácený o méně důležité informace)

Webová aplikace je nyní spuštěna v Docker kontejneru a je dostupná na adrese `http://localhost:3000`.

3.4 Kontejnerizace backendové části aplikace

3.4.1 Vytvoření Dockerfile

Pro backendovou část aplikace je nutné vytvořit další Dockerfile, tentokrát v kořenovém adresáři backendové části, tedy `backend`. Stejně jako u frontendu můžeme využít

víceúrovňové sestavení obrazu pro optimalizaci a oddělení vývojového a produkčního prostředí. Zde je ukázka první části Dockerfile:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

První stádium je identické s prvním stádiem frontendového Dockerfile. Stáhne se obraz Node.js, nainstalují se závislosti a sestaví se aplikace.

```
FROM node:20-alpine
COPY --from=build /app/dist ./dist
COPY package*.json ./
RUN npm install --only=production
EXPOSE 3001
CMD [ "npm", "run", "start" ]
```

Druhá část souboru definuje produkční obraz backendové části aplikace. Zde se opět používá obraz Node.js, tentokrát ale pouze pro běh aplikace. To znamená, že není nutné používat další webový server jako nginx, protože backendová část nepotřebuje obsluhu statických souborů prohlížečem, jako tomu bylo v předešlé kapitole. Backendová část aplikace poskytuje API, které komunikuje s frontendovou částí a poskytuje jí data a funkce. Tato komunikace probíhá přes požadavky HTTP(S), ale není potřeba poskytovat statický obsah prohlížeči. Node.js se pro tyto účely často používá, a proto je vhodný pro běh backendové části aplikace. Následuje instrukce kopírování sestaveného kódu do adresáře `dist`, instalace produkčních závislostí a je definován port, na kterém kontejner naslouchá, tedy 3001. Poslední instrukce definuje výchozí příkaz, který se má spustit při startu kontejneru. V tomto případě spouští Node.js server.

3.4.2 Sestavení obrazu a spuštění kontejneru

Sestavení obrazu a spuštění kontejneru je teoreticky hodně podobné jako v předchozí kapitole. Je zde ale jeden velký rozdíl, tahle část aplikace potřebuje ke svému fungování běžící databázi MongoDB. Sestavení obrazu z Dockerfile je stejné:

```
docker build -t backend .
```

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
backend	latest	8974d1ece33a	10 seconds ago	154MB
frontend	latest	6068099ff670	10 minutes ago	41.5MB

Pokud ale spustíme kontejnery tímto způsobem, kde `mongo` je oficiální obraz MongoDB z Docker Hubu:

```
docker run -d mongo:7.0.6
docker run -d -p 3001:3001 backend
```

tak spolu kontejnery nemohou komunikovat, protože jsou oba kontejnery oddělené, každý ve svém vlastním prostředí a běží v oddělených síťových prostorech. Možností řešení tohoto problému je několik. Ideální je použití kontejnerového orchestrátoru, kde se všechny kontejnery sestaví a spustí v rámci jednoho clusteru. To umožňuje vzájemnou komunikaci kontejnerů bez nutnosti další konfigurace. Toto řešení bude popsáno v následující sekci. Pokud ale uživatel z nějakého důvodu nechce používat orchestrátor, lze použít příkaz `docker network`, který umožňuje manuální konfiguraci síťového rozhraní kontejnerů. Použití je vidět v následující ukázce:

```
docker network create db-network
docker run -d --network db-network mongo:7.0.0
docker run --network db-network -e MONGODB_URI=mongodb://mongodb:27017/ToDoList \
  -d -p 3001:3001 backend
docker ps
```

CONTAINER ID	IMAGE	STATUS	...
98557ffa3d30	backend	Up 37 seconds	...
fba25e736b50	mongo:7.0.0	Up About a minute	...
30168696ab53	frontend	Up 15 minutes	...

Po použití příkazu `docker ps` jde vidět, že oba kontejnery běží. (výpis je zkrácený o méně důležité informace) Parametr `-e` určuje proměnnou pro prostředí kontejneru. Jeho použití je potřebné, protože soubor `.env`, ve kterém se proměnné definují, je v souboru `.dockerignore`.

3.5 Automatizace sestavení obrazů a spuštění kontejnerů

Sestavování obrazů a spouštění kontejnerů jeden po druhém může být únavné a zdlouhavé. Jednoduchý způsob automatizace tohoto procesu je použití kontejnerového orchestrátoru Docker Compose. Ten umožňuje definovat a spustit všechny kontejnery aplikace jediným příkazem. Místo manuálního spouštění kontejnerů si je potřeba vytvořit konfigurační soubor `docker-compose.yml`, ve kterém jsou popsány jednotlivé služby aplikace. V následující ukázce je vytvořený soubor `docker-compose.yml`, který se skládá ze tří služeb – frontend, backend a mongodb.

```

services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - VITE_API_URL=http://localhost:3001
    container_name: frontend

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "3001:3001"
    depends_on:
      - mongodb
    environment:
      - PORT=3001
      - MONGODB_URI=mongodb://mongodb:27017/ToDoList
    container_name: backend

  mongodb:
    image: mongo:7.0.0
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
    container_name: mongodb

volumes:
  mongodb_data:

```

Konfigurace služeb frontend a backend jsou podobné, sekce `build` specifikuje, jak vytvořit obraz dané služby, tedy kde najít soubory služby a jak se nazývá Dockerfile. Sekce `environment` definuje proměnné prostředí použité v kódu. U služby backend je navíc sekce `depends_on`, ta zaručuje, že se služba spustí až po službě mongodb. Služba mongodb je sestavena z oficiálního obrazu mongo. Sekce `volumes` připojí svazek do adresáře uvnitř kontejneru pro data mongodb, svazek je definován na konci souboru. To zajistí, že data databáze zůstanou zachována, i když je kontejner znovu vytvořen nebo zastaven. Nyní je možné sestavit a spustit všechny kontejnery jediným příkazem:

```

docker compose up -d
docker compose ps

```

NAME	IMAGE	SERVICE	CREATED	STATUS	...
backend	projekt-backend	backend	About a minute ago	Up 47 seconds	...

frontend	projekt-frontend	frontend	About a minute ago	Up 48 seconds ...
mongodb	mongo:7.0.0	mongodb	About a minute ago	Up 47 seconds ...

Po použití příkazu `docker compose ps` jde vidět, že všechny kontejnery běží. (výpis je zkrácený o méně důležité informace). Pro zastavení kontejnerů lze použít příkaz `docker compose down`. Tímto způsobem bylo zautomatizováno sestavení obrazů a spouštění kontejnerů.

3.6 Ukázka instalace kontejnerizované aplikace

V této části bude znázorněno, jak jednoduše nainstalovat a spustit kontejnerizovanou aplikaci pomocí Dockeru. Pro tuto ukázkou bude využit veřejně dostupný repozitář na GitHubu obsahující aplikaci popsanou v této kapitole. Pro instalaci celého projektu stačí do příkazové řádky zadat pouze tři příkazy:

```
git clone https://github.com/OndraMarek/ToDo-List.git
cd ToDo-List
docker compose up -d
```

Tímto způsobem bude aplikace nainstalována bez nutnosti instalace dalších nástrojů, jako jsou například Node.js nebo MongoDB. Díky použití kontejnerizačního řešení Docker se veškeré potřebné závislosti a prostředí spravují automaticky. Uživatel není nucen řešit složité konfigurace či instalace, neboť Docker zajistí kompletní izolaci a správu prostředí potřebného pro běh aplikace.

Závěr

Všechny předpokládané cíle práce byly splněny. Čtenář byl seznámen s problematikou kontejnerizačních technologií. Kromě popisu technologií práce obsahuje i názorné obrázky a zdrojové kódy, které pomáhají danou problematiku pochopit.

V první části práce byly pečlivě prozkoumány a analyzovány principy a technologie kontejnerizace, se zaměřením na klíčové aspekty jako agilita, přenositelnost, rychlost a škálovatelnost. Dále bylo provedeno srovnání kontejnerizace s virtualizací a představeny různé kontejnerové enginy a orchestrace, včetně populárních nástrojů jako Docker, Podman, CRI-O, containerd, Docker Compose, Docker Swarm a Kubernetes.

Druhá část práce se pak zaměřila na detailní popis technologie Dockeru. Čtenář byl proveden architekturou Docker enginu a seznámil se s jednotlivými komponentami tohoto prostředí, včetně Docker Daemon, containerd, runc, Docker Client, Docker Host, Docker Registry a Docker Objects, tedy obrazy a kontejnery. Dále byl popsán proces sestavování obrazů pomocí Dockerfile a jeho základními instrukcemi. Také byl čtenář seznámen se základními příkazy pro práci s Dockerem, které umožňují uživatelům vytvářet, spravovat a provozovat kontejnery a obrazy. Nakonec byly rozebrány klíčové aspekty bezpečnosti Dockeru a strategie pro minimalizaci rizik spojených s provozem kontejnerů a správou obrazů.

Poslední část práce přinesla konkrétní aplikaci teoretických poznatků v praxi prostřednictvím tvorby kontejneru pro specifickou aplikaci. Podrobně byl popsán proces vytvoření kontejneru včetně tvorby Dockerfile, sestavení Docker obrazů a kontejnerů a jejich následného spuštění pomocí Docker Compose. Tato část práce zdůraznila flexibilitu a přenositelnost Dockeru, která umožňuje spouštění aplikací z různých prostředí. Konkrétní postup vytvoření kontejneru a jeho nasazení byl demonstrován na reálném příkladu, což pomůže čtenářům lépe pochopit praktické aplikace kontejnerizace.

Seznam použité literatury

- [1] *Cloud Computing Services - Amazon Web Services (AWS): What's the Difference Between Docker Images and Containers?* Online. AMAZON, INC. Dostupné z: <https://aws.amazon.com/compare/the-difference-between-docker-images-and-containers/>. [cit. 2023-1-27].
- [2] *Docker Docs*. Online. DOCKER, INC. Dostupné z: <https://docs.docker.com/>. [cit. 2023-12-20].
- [3] *Docker Docs: Install Docker Desktop on Mac*. DOCKER, INC. Online. Dostupné z: <https://docs.docker.com/desktop/install/mac-install/>. [cit. 2024-03-25].
- [4] *Docker Docs: Install Docker Desktop on Windows*. Online. DOCKER, INC. Dostupné z: <https://docs.docker.com/desktop/install/windows-install/>. [cit. 2024-03-25].
- [5] *Docker Docs: Install Docker Engine on Ubuntu*. Online. DOCKER, INC. Dostupné z: <https://docs.docker.com/engine/install/ubuntu/>. [cit. 2024-03-25].
- [6] *Docker Docs: Use containers to Build, Share and Run your applications Online*. DOCKER, INC. Dostupné z: <https://www.docker.com/resources/what-container/>. [cit. 2023-12-18].
- [7] *Home - taikun.cloud: How to Handle Container Storage*. Online. TAIKUN.CLOUD A.S. Dostupné z: <https://taikun.cloud/how-to-handle-container-storage/>. [cit. 2023-1-27].
- [8] HYKES, Solomon. *Introducing runC: a lightweight universal container runtime*. Online. 2015. Dostupné z: <https://www.docker.com/blog/runc/>. [cit. 2023-1-25].
- [9] *IBM - United States: What is containerization?* Online. IBM. Dostupné z: <https://www.ibm.com/topics/containerization>. [cit. 2023-12-19].
- [10] *IBM - United States: What is container orchestration?* Online. IBM. Dostupné z: <https://www.ibm.com/topics/container-orchestration>. [cit. 2023-12-19].
- [11] *Kubernetes Documentation / Kubernetes*. Online. Dostupné z: <https://kubernetes.io/docs/home/>. [cit. 2023-12-21].
- [12] KUZMENKO, Evgenia. *How Do the Docker Client and Docker Servers Work?* Online. 2023. Dostupné z:

- <https://dzone.com/articles/how-do-the-docker-client-and-docker-serves-work>. [cit. 2023-1-26].
- [13] LUKIC, Djordje. *Extending Docker's Integration with containerd*. Online. 2022. Dostupné z: <https://www.docker.com/blog/extending-docker-integration-with-containerd/>. [cit. 2023-1-24].
- [14] MCCABE, John a FRIIS, Michael. *Introduction to Windows Containers*. Online. 2017. Dostupné z: https://learn.microsoft.com/en-us/archive/blogs/microsoft_press/free-ebook-introduction-to-windows-containers. [cit. 2023-12-19].
- [15] *Mezmo: 6 Best Kubernetes Distributions: Openshift, Rancher, AKS, EKS, GKE and DigitalOcean*. Online. MEZMO INC. Dostupné z: <https://www.mezmo.com/learn-kubernetes/6-best-kubernetes-distributions-openshift-rancher-aks-eks-gke-and-digitalocean>. [cit. 2023-12-21].
- [16] *MongoDB: The Developer Data Platform / MongoDB: MongoDB Documentation*. Online. MONGODB, INC. Dostupné z: <https://www.mongodb.com/docs/>. [cit. 2024-3-24].
- [17] *Node.js — About Node.js®*. Online. OPENJS FOUNDATION. Dostupné z: <https://nodejs.org/en/about>. [cit. 2024-3-24].
- [18] *Open Container Initiative - Open Container Initiative*. Online. THE LINUX FOUNDATION. Dostupné z: <https://opencontainers.org/>. [cit. 2023-1-26].
- [19] *Podman*. Online. PODMAN. Dostupné z: <https://podman.io/>. [cit. 2023-12-20].
- [20] POULTON, Nigel. *Docker Deep Dive*. 2023 Edition. 2023. ISBN 978-1521822807.
- [21] *React: Quick Start - React*. Online. META PLATFORMS, INC. Dostupné z: <https://react.dev/learn>. [cit. 2024-3-24].
- [22] RUSSO, Stefano Alberto. *Container engines, runtimes and orchestrators: an overview*. Online. 8.2.2022. Dostupné z: <https://sarusso.github.io/blog/container-engines-runtimes-orchestrators.html>. [cit. 2023-12-20].
- [23] SARKAR, Avijit. *Docker and OCI Runtimes*. Online. 5.11.2019. Dostupné z: <https://medium.com/@avijitsarkar123/docker-and-oci-runtimes-a9c23a5646d6>. [cit. 2023-12-20].

- [24] STRONGLOOP. *Express - Node.js web application framework*. Online.
Dostupné z: <https://expressjs.com/>. [cit. 2024-3-24].
- [25] VERNEROVÁ, Sára. *Front end vs. Back end - jaký je mezi nimi rozdíl?* Online.
Dostupné z:
<https://www.apitree.cz/blog/front-end-vs-back-end-jaky-je-mezi-nimi-rozdil>. [cit. 2024-3-24].
- [26] *Vite / Next Generation Frontend Tooling: Getting Started / Vite* Online.
Dostupné z: <https://vitejs.dev/guide/>. [cit. 2024-3-24].

Seznam obrázků

1	Porovnání VM a kontejnerizace	5
2	Diagram vztahů mezi orchestrátory, enginy a runtime	6
3	Komunikace mezi klientem, daemonem, containerd a runc	12
4	Komunikace mezi klientem, daemonem, containerd a runc	13
5	Architektura Dockeru	15
6	Vrstvy kontejnerů	17
7	Ukázka To-Do List aplikace	25

Seznam zkratek

API	Application Programming Interface
CD	Continuous Delivery
CI	Continuous Integration
CLI	Command Line Interface
CPU	Central Processing Unit
CRI-O	container runtime interface for OCI
DoS	Denial of Service
GB	Gigabyte
GPG	GNU Privacy Guard
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
LTS	Long Term Support
MB	Megabyte
OCI	Open Container Initiative
PaaS	Platform as a Service
RAM	Random Access Memory
REST	Representational State Transfer
SSH	Secure Shell
UI	User Interface
URL	Uniform Resource Locator
VM	Virtual Machine
YAML	YAML Ain't Markup Language

PŘÍLOHY

- Elektronické přílohy:
 - FPF_BP_2024_Kontejnerizace_aplikace_Marek_Ondrej.pdf
 - kontejnerizovana_webova_aplikace.zip